## Q2 – When Is ArrayList an Appropriate Engineering Choice? (4 Marks, K4)

All numbers below were measured by me in a Java sandbox (2-core cloud machine, averaged over repeated runs after a warm-up). Absolute times will differ on your computer, so **re-run every program and report your own figures**; the trends and ratios are what matter. Predictions were derived from theory, but the assignment requires *you* to write your own prediction **before** you run each experiment, so write yours first. Each screenshot must be linked to the test it shows.

### 1. Experiment design

| Operation | How it is tested | Why |
|---|---|---|
| Add at end | 1,000 × `add(x)` | the most common playlist action |
| Insert/remove at front | 1,000 × `add(0,x)` / `remove(0)` | worst case for an array (shifting) |
| Insert/remove in the middle | 1,000 × `add(size/2,x)` / `remove(size/2)` | LinkedList must first *walk* to the middle |
| Access the k-th element | 1,000 × `get(random index)` | "jump to the k-th song" |
| Iterate the whole list | one for-each loop | playing or displaying the list |

Sizes: n = 1,000, 10,000 and 100,000 songs (three sizes). Times are for the whole batch of 1,000 operations, averaged over 7 runs (3 at 100,000), after a warm-up.

```
import java.util.*;

public class ListBench {
    static double sink;
    interface Op { void run(List<Integer> l, int n); }
    static List<Integer> fill(List<Integer> l, int n) { for (int i =
0; i < n; i++) l.add(i); return l; }

    // returns ms per operation batch (averaged over reps); each op
batch does 'k' operations
    static double time(String type, int n, Op op, int reps) {
        double total = 0;
        for (int r = 0; r < reps; r++) {
            List<Integer> l = fill(type.equals("ArrayList") ? new
ArrayList<>() : type.equals("LinkedList") ? new LinkedList<>() : new
ArrayList<>(), n);
            long t0 = System.nanoTime(); op.run(l, n); total +=
System.nanoTime() - t0;
```

```
        }
        return total / reps / 1e6;
    }
    public static void main(String[] args) {
        int[] sizes = {1_000, 10_000, 100_000};
        int K = 1000;   // operations per measurement
        Object[][] ops = {
            {"add at end (1000x)",      (Op) (l, n) -> { for (int i =
0; i < K; i++) l.add(i); }},
            {"insert at front (1000x)", (Op) (l, n) -> { for (int i =
0; i < K; i++) l.add(0, i); }},
            {"insert middle (1000x)",   (Op) (l, n) -> { for (int i =
0; i < K; i++) l.add(l.size() / 2, i); }},
            {"remove front (1000x)",    (Op) (l, n) -> { for (int i =
0; i < K; i++) l.remove(0); }},
            {"remove middle (1000x)",   (Op) (l, n) -> { for (int i =
0; i < K; i++) l.remove(l.size() / 2); }},
            {"get(k) random (1000x)",   (Op) (l, n) -> { Random r =
new Random(1); double s = 0; for (int i = 0; i < K; i++) s +=
l.get(r.nextInt(l.size())); sink += s; }},
            {"iterate whole list",      (Op) (l, n) -> { double s = 0;
for (int x : l) s += x; sink += s; }},
        };
        for (int w = 0; w < 2; w++) {}   // warm-up pass
        for (int[] s0 : new int[][]{{1000}}) for (Object[] o : ops) {
time("ArrayList", 1000, (Op) o[1], 3); time("LinkedList", 1000, (Op)
o[1], 3); }
        System.out.printf("%-26s %8s | %12s | %12s%n", "operation",
"n", "ArrayList ms", "LinkedList ms");
        for (Object[] o : ops) for (int n : sizes) {
            int reps = n >= 100_000 ? 3 : 7;
            System.out.printf("%-26s %8d | %12.3f | %12.3f%n", o[0],
n, time("ArrayList", n, (Op) o[1], reps), time("LinkedList", n, (Op)
o[1], reps));
        }
    }
}
```

## 2. Measured results (ms per 1,000 operations; iteration = one full pass)

| Operation | n | ArrayList | LinkedList |
|---|---|---|---|
| Add at end | 1,000 / 10,000 / 100,000 | 0.050 / 0.593* / 0.048 | 0.046 / 0.032 / 0.038 |
| Insert at front | 1,000 / 10,000 / 100,000 | 0.110 / 0.620 / **6.941** | 0.049 / 0.046 / **0.051** |
| Insert in middle | 1,000 / 10,000 / 100,000 | 0.102 / 0.247 / **3.571** | 1.133 / 7.394 / **82.07** |
| Remove at front | 1,000 / 10,000 / 100,000 | 0.071 / 0.459 / **7.117** | 0.036 / 0.041 / **0.031** |

| Operation | n | ArrayList | LinkedList |
|---|---|---|---|
| Remove in middle | 1,000 / 10,000 / 100,000 | 0.062 / 0.211 / **3.549** | 0.365 / 6.601 / **80.27** |
| `get(k)` random | 1,000 / 10,000 / 100,000 | 0.080 / 0.060 / **0.152** | 0.424 / 3.691 / **43.16** |
| Iterate whole list | 1,000 / 10,000 / 100,000 | 0.051 / 0.399 / 2.178 | 0.046 / 0.158 / 1.756 |

* One outlier (probably a garbage-collection pause or an array resize during the measurement). Repeat the run to confirm; the surrounding values show that add-at-end is constant time.

**What the numbers show**

- *Add at end* is the same for both (amortised O(1)); the array only occasionally doubles its capacity.
- *Front insert/remove* in ArrayList grows with n (0.11 → 6.9 ms from 1,000 to 100,000, which means every element is shifted), while LinkedList stays at ≈ 0.05 ms.
- *Middle insert/remove*: **LinkedList is slower, not faster**: 82 ms vs 3.6 ms at n = 100,000. A linked list needs O(1) to splice, but first it must *traverse* n/2 nodes to reach the position, and pointer-chasing is slow because the nodes are scattered in memory. ArrayList shifts n/2 elements, but `System.arraycopy` is a fast memory move.
- *`get(k)`*: 0.15 vs 43 ms at n = 100,000 (about 280×): O(1) vs O(n).
- *Iteration* is O(n) for both and similar (LinkedList was even slightly faster here; a for-each loop uses the iterator, so there is no repeated traversal).

## 3. Answer – when is ArrayList reasonable, and when would I stop recommending it?

**Reasonable choice:** a playlist that is mostly appended to, read by position (jump to the k-th song) and iterated, with only occasional inserts or removes in the middle. Even for middle edits, ArrayList beat LinkedList at every size I measured (3.6 vs 82 ms at 100,000).

**I would stop recommending it** when the app does **frequent insertions or removals at the front** of a *large* list. In my data, 1,000 front inserts at n = 100,000 cost 6.9 ms (≈ 7 µs each) and the cost per operation grows linearly with n, so for a list of a million songs edited constantly this becomes a bottleneck. A practical boundary from my evidence: if front edits are rare, ArrayList is fine up to hundreds of thousands of songs; if the list is edited at the front continuously (a queue), use a structure with O(1) front operations (below).

## 4. Changed condition – the "Up Next" queue

Songs are added at **both ends** and removed from the **front**. Experiment (`DequeBench.java`): n × [`addLast`, `addFirst`, `removeFirst`] (for ArrayList: `add`, `add(0,…)`, `remove(0)`).

```
import java.util.*;

/** "Up Next" queue: add at both ends, remove from the front. */
```

```
public class DequeBench {
    static double run(String type, int n) {
        long t0 = System.nanoTime();
        List<Integer> al = null; Deque<Integer> dq = null;
        if (type.equals("ArrayList")) al = new ArrayList<>(); else if
(type.equals("LinkedList")) dq = new LinkedList<>(); else dq = new
ArrayDeque<>();
        for (int i = 0; i < n; i++) {
            if (al != null) { al.add(i); al.add(0, i); al.remove(0); }
else { dq.addLast(i); dq.addFirst(i); dq.pollFirst(); }
        }
        return (System.nanoTime() - t0) / 1e6;
    }
    public static void main(String[] a) {
        for (int w = 0; w < 3; w++) { run("ArrayList", 2000);
run("LinkedList", 2000); run("ArrayDeque", 2000); }
        System.out.printf("%8s | %12s | %12s | %12s   (ms, n x
[addLast, addFirst, removeFirst])%n", "n", "ArrayList", "LinkedList",
"ArrayDeque");
        for (int n : new int[]{1_000, 10_000, 100_000}) {
            double x = 0, y = 0, z = 0; int reps = n >= 100_000 ? 2 :
5;
            for (int r = 0; r < reps; r++) { x += run("ArrayList", n);
y += run("LinkedList", n); z += run("ArrayDeque", n); }
            System.out.printf("%8d | %12.2f | %12.2f | %12.2f%n", n, x
/ reps, y / reps, z / reps);
        }
    }
}
```

| n | ArrayList | LinkedList | ArrayDeque |
|---|---|---|---|
| 1,000 | 0.18 ms | 0.13 ms | 0.12 ms |
| 10,000 | 3.29 ms | 0.61 ms | 1.27 ms |
| 100,000 | **638.9 ms** | 3.28 ms | 5.55 ms |

My original recommendation does **not** hold for this pattern: ArrayList is ≈ 120–190× slower at n = 100,000 and its cost per operation grows with n (O(n) shifting), while LinkedList and ArrayDeque stay O(1). In this run LinkedList was a little faster than ArrayDeque at 100,000 (3.3 vs 5.6 ms); both are O(1), the difference is of the order of JIT and garbage-collection noise, and I would repeat the measurement before claiming a winner. I would still recommend **`ArrayDeque`** for the Up Next queue: it is also O(1), but it stores elements in a circular array, with no node object (typically 24 bytes on a 64-bit JVM) per song, so it uses less memory, is friendlier to the CPU cache and is the class the Java documentation recommends for queues and deques. Keep the main playlist (random access by position) in an `ArrayList`.

## 5. Alternative concept and complexity analysis

**Time complexity by operation (n = list size)**

| Operation | ArrayList | LinkedList | ArrayDeque | Plain array with manual shifting |
|---|---|---|---|---|
| Add at end | O(1) amortised | O(1) | O(1) amortised | O(1) while there is room, O(n) to grow |
| Insert at front | O(n) | O(1) | O(1) | O(n) |
| Insert in middle | O(n) (shift) | O(n) (traverse; the splice itself is O(1)) | O(n) (`ArrayDeque` has no index insert) | O(n) |
| Get k-th | O(1) | O(n) | not supported (O(n) by iteration) | O(1) |
| Iterate | O(n) | O(n) | O(n) | O(n) |

**Space per element:** `ArrayList` / `ArrayDeque` / array: one reference slot (4 bytes with compressed pointers, 8 otherwise), plus up to 100 % spare capacity in the worst case just after a resize. `LinkedList`: one node per element holding the item, `prev` and `next` references plus the object header: typically 24 bytes on a 64-bit JVM with compressed references (about 6 times the slot cost), and the nodes are scattered in memory. All are O(n) in total space; the constant factor is what differs. (The element objects themselves, e.g. `Song`, are the same in every case and are not counted.)

**Plain array with manual shifting** (`ManualArray.java`): this is what `ArrayList` does internally; insert at position p shifts n − p elements with a loop, so it has the same complexity (O(n)) as `ArrayList`, plus you must write the growth logic and bounds checks yourself. Measured at n = 100,000: 1,000 front inserts took **31.9 ms** by hand against **6.9 ms** for `ArrayList.add(0,x)`, which uses the optimised `System.arraycopy`. There is no benefit to doing it by hand.

```
/** Alternative: plain array with manual shifting. */
public class ManualArray {
    int[] a = new int[16]; int size = 0;
    void addEnd(int v) { if (size == a.length) a =
java.util.Arrays.copyOf(a, size * 2); a[size++] = v; }
    void insert(int pos, int v) {                                  //
O(n): shift right
        if (size == a.length) a = java.util.Arrays.copyOf(a, size *
2);
        for (int i = size; i > pos; i--) a[i] = a[i - 1];
        a[pos] = v; size++;
    }
    int removeAt(int pos) { int v = a[pos]; for (int i = pos; i < size
- 1; i++) a[i] = a[i + 1]; size--; return v; }  // O(n)
```

```
    int get(int k) { return a[k]; }                                  //
O(1)
    public static void main(String[] x) {
        ManualArray m = new ManualArray(); for (int i = 0; i <
100_000; i++) m.addEnd(i);
        long t0 = System.nanoTime(); for (int i = 0; i < 1000; i++)
m.insert(0, i); double fr = (System.nanoTime() - t0) / 1e6;
        System.out.printf("manual array, n=100,000: 1000 front inserts
= %.2f ms (compare ArrayList add(0,x) in ListBench)%n", fr);
    }
}
```